Points Tracker Architecture Plan v2.1

Oct 24, 2026 · @Stanley

What changed in v2.1

v2.1 replaces uv with **pip + venv**, the default in most enterprise Python teams, so the tooling skills transfer to other jobs. Everything else in v2 is unchanged.

Job	v2 (uv)	v2.1 (pip)
Virtual environment	`uv` creates `.venv`	`python -m venv .venv`
Install packages	`uv add`	`pip install -e .`
Pin exact versions	`uv.lock`	`requirements.txt` from `pip freeze`
Build backend for the `dev` entry point	`uv_build`	`setuptools`
Python version	`.python-version` + `uv python install`	Windows `py` launcher

What changed from v1

v2 keeps v1's core design: a Postgres ledger, a FastAPI backend, and Claude ranking options that code computes. It changes the build order, fixes data-model gaps, and names the tooling. v1 was prepared October 2, 2026; this version replaces it.

Change	Why
Phase 1 split into **1a manual entry** and **1b AwardWallet sync**	The riskiest outside dependency no longer blocks a usable app. You learn the ledger before an outside system writes to it.

Change	Why
Snapshot-first reconciliation	Many programs return thin or no activity history, so ledger sum vs. snapshot would drift constantly. Gaps now become explicit `adjust` events linked to the snapshot that revealed them.
Data model gaps fixed	Removed the `snapshot` event kind; added reversals, transfer grouping, a CSV review table, strict units, alert dedupe, and `optimize_runs`.
Plain view instead of materialized view	Always current, with no refresh step to forget. Materialize later only if measured to be slow.
Jobs live inside `api/` with their own entrypoint	Workers share models, settings, and database code with the API.
Tooling named: pip + venv, pydantic-settings, pytest, generated TypeScript client	Reproducible installs, typed config, tests against real Postgres, and frontend types that cannot drift from the API.
Claude optimizer refinements	Placeholder-only figures, structured outputs, an audit and cache table, model ID in config.
Redis kept, RQ recommended	An explicit learning goal. Introduced in Phase 1b, when there is first real background work.

The product

Points Tracker treats every points balance you hold as one portfolio and recommends the cheapest points-or-cash path for one trip at a time. The stack is Python (FastAPI), Postgres, Redis, React, the Claude API, and the AwardWallet API. The primary user is Stanley, and it doubles as a learning project, so some tools are chosen for what they teach.

Most points tools track one program, so value leaks across programs. Chase Ultimate Rewards sit idle while a Hyatt stay could absorb them at 2+ cents per point, or airline miles expire while a transfer bonus is active. Balances start as manual entries, then sync from AwardWallet. There is no email parsing and no bank password in this app.

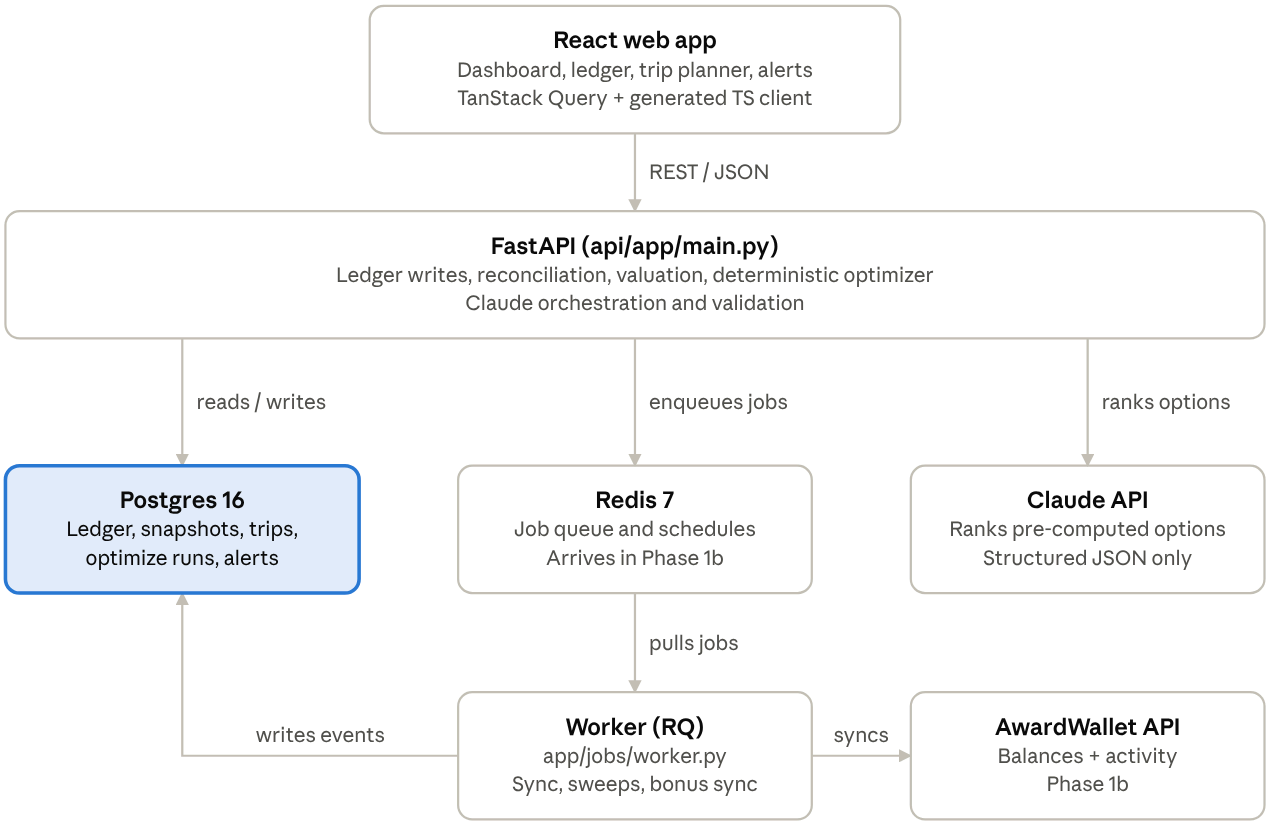
You enter a trip, and code enumerates every payment option, scored in dollars saved and cents per point (cpp) realized. Claude then ranks those options and explains the trade-offs, with the math shown, not guessed.

Design principle: never overwrite a balance. Every earn, burn, transfer, expiration, and correction is a ledger event. Undo is a new event that reverses an old one; nothing is ever updated or deleted.

System architecture

The system has a React frontend, a FastAPI backend, a Postgres ledger, and a Redis-backed worker that runs the same Python package as the API. Claude sits behind the API as a ranking service and never computes balances or prices.

One Postgres ledger; the worker reuses the API code through Redis



system architecture · 6 components

The API and the worker are one Python package with two entrypoints; Redis carries jobs between them, and both write to the same Postgres ledger.

Component	Tech	Responsibility
Web app	React + Vite + TypeScript, TanStack Query, a router, generated API client	Dashboard, ledger, trip planner, alert inbox, settings
API	Python 3.12+, FastAPI, SQLAlchemy 2.0 (sync), Alembic, pydantic-settings	CRUD, ledger writes, valuation engine, deterministic optimizer, Claude orchestration
Database	Postgres 16	Ledger events, snapshots, program catalog, valuations, trips, optimize runs, alerts
Queue	Redis 7	Job queue and scheduled-job storage for the worker
Worker	RQ (recommended) or Celery, same codebase as the API	AwardWallet sync, expiry sweeps, transfer-bonus sync
Tooling	pip + venv, setuptools, pytest, openapi-typescript	Virtual environment and pinned dependencies, tests, frontend type generation
Balance source	Manual and CSV (1a); AwardWallet Account Access API (1b)	Balance snapshots and activity history
AI	Claude API with structured outputs	Ranks pre-computed options and explains them; no ledger writes, no invented figures

Redis and the worker: RQ vs. Celery

RQ is recommended because it is small enough to understand fully and keeps Redis visible.

	RQ (recommended)	Celery
Learning curve	Small API; you can read its source in an afternoon. Jobs are plain functions.	Many concepts: brokers, result backends, beat, routing, canvas. More to learn and to misconfigure.

	RQ (recommended)	Celery
Redis exposure	Redis-only, so you can inspect the queue directly with `redis-cli`.	Redis is one of several brokers, with more abstraction between you and Redis.
Scheduling	RQ's scheduler or `rq-scheduler` for repeating jobs	Celery beat, mature and full-featured
Industry use	Common in smaller apps	The default in larger Python teams

Windows note: neither runs cleanly on native Windows. RQ workers need `fork()`, and Celery no longer officially supports Windows. Run the worker in Docker or WSL, and run the API and Vite natively for fast reloads.

Authentication and credential storage

Phase 1 has no login system: the app runs behind your home network or Tailscale, or behind HTTP Basic Auth on the reverse proxy. This is unchanged from v1.

- **If it becomes multi-user:** store only argon2id password hashes, and use server-side sessions in an `HttpOnly`, `Secure`, `SameSite` cookie.
- **Never** store JWTs in `localStorage`; any cross-site scripting bug can read them.

Credential	Stored where	Protection
AwardWallet OAuth tokens (1b)	`credentials` table	Encrypted at rest (Fernet or AES-GCM). The key is an environment variable, loaded as a `SecretStr` by pydantic-settings. Narrowest scope.
AwardWallet API key (1b)	Environment or git-ignored `.env`	IP-restricted, server-side only, never committed
Claude API key	Environment or git-ignored `.env`	Server-side only; the browser never calls Claude
Bank and loyalty passwords	Never in this app	Held by AwardWallet only if you link programs there; otherwise manual or CSV entry

The tradeoff: automatic balances require someone to log in to Chase, Hyatt, and the rest. AwardWallet does that on its side, so your program credentials live there, not here. Manual and CSV entry remain for anything you would rather not delegate.

Data model: an event-sourced ledger

Balances are derived from the ledger, never stored as a mutable number. Changes from v1 are marked **(new)** or **(changed)**.

Units: one rule for every table (new)

- **Points:** `BIGINT`, signed. Positive adds to a balance; negative removes from it.
- **Money:** `BIGINT` integer cents, e.g. `cash_price_cents`.
- **Cents per point:** `NUMERIC(6,3)`, e.g. `1.850`. Never floats.
- **Moments:** `timestamptz`. **Calendar days** (expiry, effective dates): `date`.

Core tables

Table	Key columns	Notes
`programs`	id, name, kind (bank / hotel / airline), currency_name, expiry_rule	Data rows, not code
`accounts`	id, user_id, program_id, member_ref (last 4), status, expires_at (date), awardwallet_account_id (nullable, unique) **(changed)**	One row per loyalty account
`ledger_events`	id, account_id, kind, points, occurred_at, recorded_at, source, external_ref, note, transfer_group_id **(new)**, reverses_event_id **(new)**, snapshot_id **(new)**	Append-only. Kinds: earn, burn, transfer_in, transfer_out, expire, adjust, reversal. The `snapshot` kind is removed **(changed)**.
`balance_snapshots`	id, account_id, points, captured_at, source, external_ref	Point-in-time totals from AwardWallet or a manual check-in

Table	Key columns	Notes
`import_rows` **(new)**	id, batch_id, raw (jsonb), parsed fields, status (pending / approved / rejected), ledger_event_id	The CSV review queue v1 described but never modeled
`transfer_partners`	id, from_program_id, to_program_id, ratio_num, ratio_den, active, bonus_pct (numeric), bonus_ends_at	Ratios and bonuses as data
`valuations`	program_id, cpp `NUMERIC(6,3)`, effective_date, source **(changed)**	Renamed from `cpp_cents`, since cpp is already in cents
`trips` / `trip_legs`	trip: origin, destination, dates, travelers; leg: kind, cabin, nights, cash_price_cents, price_captured_at **(new)**	Prices go stale, so record when they were seen
`optimize_runs` **(new)**	id, trip_id, input_hash, payload (jsonb), model_id, response (jsonb), validation_status, created_at	Audit trail and cache for every Claude call
`redemption_options`	optimize_run_id **(new)**, option_key, trip_leg_id, program_id, points_cost, cash_copay_cents, cpp_realized, rank, irreversible, verify_availability	Tied to the run that produced them, so old and new options never mix
`alerts`	id, user_id, account_id, kind, payload, dedupe_key (unique) **(new)**, created_at, seen_at	Daily sweeps cannot create duplicates

Constraints that make the ledger trustworthy (new)

- `CHECK` that the sign matches the kind: earn and transfer_in are positive; burn, transfer_out, and expire are negative; adjust and reversal are non-zero.
- `CHECK` that transfer kinds have a `transfer_group_id`. A transfer is two events, `transfer_out` from Chase and `transfer_in` to Hyatt, written in one database transaction.

- `reverses_event_id` is `UNIQUE`, so an event can be undone only once. A reversal carries the opposite points.
- A unique `(source, external_ref)` where `external_ref` is set, so re-syncs and re-imports are idempotent.
- Append-only is enforced in the database: the app's role has no `UPDATE` or `DELETE` on `ledger_events`.

Snapshot-first reconciliation (changed)

Each AwardWallet sync, or manual "is this still right?" check-in, runs these steps in order:

1. Import new activity as ledger events, deduped on `source` + `external_ref`.
2. Write the snapshot to `balance_snapshots`.
3. Compute the gap: snapshot points minus the sum of the account's ledger events.
4. If the gap is non-zero, write one `adjust` event for the gap, with `source = reconciliation` and `snapshot_id` set to the snapshot that revealed it.
5. If the account normally has activity history and the gap exceeds its tolerance, raise an alert: "was this yours?"

Why this order works:

- The dashboard shows one number you can trust, and every correction traces back to the snapshot that caused it.
- If activity arrives late, after an adjust already covered the gap, the next sync finds the ledger too high and writes an opposite adjust. It self-corrects, and the history shows exactly what happened.
- The same code path serves manual check-ins in Phase 1a and AwardWallet in Phase 1b.

Plain view vs. materialized view (changed)

`account_balances` is a plain view that sums `ledger_events` per account, backed by an index on `ledger_events(account_id)`.

	Plain view (chosen)	Materialized view
What it is	A saved query; Postgres runs it every time you read from it	A saved query whose result is stored on disk like a table
Freshness	Always current	Stale until you run `REFRESH MATERIALIZED VIEW`

	Plain view (chosen)	Materialized view
Read speed	Pays the query cost on each read	Fast, since it reads stored rows
Upkeep	None	Must be refreshed after writes or on a schedule; a forgotten refresh shows a wrong balance

At personal scale, thousands of rows, a plain view is instant. Switch to a materialized view only if `EXPLAIN ANALYZE` shows it is slow.

Expiration handling: store `expires_at` per account plus the program's rule, and sweep daily for alerts at 90, 30, and 7 days, deduped by `dedupe_key`. Never silently zero a balance; an expiration is an `expire` event.

Ingestion: how balances stay current

Manual entry ships first, and AwardWallet sync follows as a second source writing the same events and snapshots. The rest of the app never knows where a number came from.

Phase 1a: manual entry and CSV import (first)

- **Opening balances:** one `adjust` event per account on day one.
- **Quick-add form:** program, points, date, kind, note. A transfer creates both of its events in one transaction.
- **Undo:** a button that writes a `reversal` event pointing at the original.
- **CSV import:** rows land in `import_rows` as `pending`; approving a row writes its ledger event.
- **Balance check-in:** enter the balance a program's site shows you. It is saved as a manual snapshot and reconciled exactly like a sync.

Phase 1b: AwardWallet sync

Gate before building: confirm AwardWallet's current API access terms, OAuth scopes, and whether an official Python client exists. v1 assumed a free business account, a `read_balances` scope, and `pip install awardwallet`; none of these has been verified. Phase 1a stays fully usable if this falls through.

- **Connect once:** OAuth2 with the narrowest scope; tokens encrypted in `credentials`.

- **Job:** `sync_awardwallet` runs on the worker on a schedule, on demand, and before an optimize run. Redis arrives in this phase.
- **Writes:** activity events (deduped), then the snapshot, then reconciliation. Re-running a sync is idempotent.
- **Cadence:** near-current, not real-time. Respect AwardWallet's refresh limits.

Deliberately not built

- No AI email parsing and no Gmail ingestion.
- No direct bank or airline APIs, and no screen-scraping.

Transfer partner and bonus sync

A curated `transfer_partners` table is updated manually or by a weekly job. Active bonuses are applied automatically and labeled "bonus ends [date]."

Valuation and the trip optimizer

Code does all the arithmetic and Claude only ranks and explains, with every figure it cites substituted in by the backend from computed values.

Valuation model

- A baseline cpp per program lives in `valuations`, editable in settings.
- **cpp realized** = (cash price - cash copay) / points cost. Options below the baseline are flagged "poor value."
- Portfolio value is shown and clearly labeled an estimate.

Optimizer flow

1. You enter a trip: legs, nights, and cash prices, each with `price_captured_at`.
2. Code enumerates options per leg: cash, burn a balance, transfer Chase points (ratio + bonus), points + cash.
3. Code scores each option: dollars saved, cpp realized, points remaining after, expiration risk.
4. Code drops clearly dominated options, assigns option keys (`opt_1` , `opt_2` , ...), and produces its own fallback ranking.

5. Code hashes the payload. If an `optimize_run` with the same `input_hash` passed validation, it is returned with no Claude call.
6. Claude receives one structured request and returns structured JSON.
7. Code validates the response, fills in placeholders, and stores the run. Any failure shows the code ranking instead.

Claude refinements (new)

Refinement	How it works and why
Placeholder-only figures	Claude refers to figures as `{opt_3.cpp}`, `{opt_3.points_cost}`, or `{opt_3.program}` and never writes digits. The backend substitutes real values. "No digits outside placeholders, and every placeholder key exists" is a simple, reliable check, unlike scanning free text for invented numbers.
Structured outputs	The response is constrained to a JSON schema: `ranked_options[]` (option_key, rank, rationale, warnings) plus `summary`. No parsing JSON out of prose.
Validation rules	The schema is valid; every option_key exists in the payload; no option is ranked twice; placeholders reference known keys and fields; no stray digits. Any failure stores `validation_status = failed` and falls back to the code ranking.
`optimize_runs` as audit and cache	Stores the payload, `input_hash`, model ID, raw response, and validation result. The hash is the cache key: when balances, prices, or bonuses change, the hash changes and a new run happens. It is also how you debug a bad ranking later.
Options tied to runs	`redemption_options.optimize_run_id` links each option to the run that produced it.
Model ID in config	`CLAUDE_MODEL` lives in pydantic-settings, so changing models is a config change. Try a smaller, cheaper model first; ranking a few pre-scored options is a modest task.

Refinement	How it works and why
Tests never call Claude	The validator is tested with recorded good and bad responses as pytest fixtures.

Rules the optimizer must respect

- Claude ranks and explains options computed from real data; it never invents prices, balances, or availability.
- All Chase transfers are labeled "irreversible" (`irreversible = true`).
- Do not spend sooner-expiring points on a later trip when an earlier use exists; surface the trade-off instead.
- Typical-cost options carry "verify availability before transferring" in both the API response and the UI.
- Cash is always an option, and sometimes it wins.

API surface and frontend

The frontend calls the API only through a TypeScript client generated from FastAPI's OpenAPI spec, so a backend change shows up as a compile error.

Endpoint	Purpose	Phase
`GET /programs`, `POST /accounts`	Catalog and accounts	1a
`GET /balances`	Derived balance per account, latest snapshot, portfolio estimate	1a
`GET /ledger?account_id=`	Event history with source and refs	1a
`POST /events`	Manual earn, burn, transfer, or adjust	1a
`POST /events/{id}/reverse` **(new)**	Undo by writing a reversal	1a
`POST /snapshots` **(new)**	Manual balance check-in; triggers reconciliation	1a
`POST /imports/csv`, `POST /imports/review`	Upload; approve or reject rows	1a

Endpoint	Purpose	Phase
`POST /awardwallet/connect`, `GET /awardwallet/status`	OAuth flow and link status	1b
`POST /awardwallet/sync`	Enqueue a sync job; returns a job ID	1b
`GET /jobs/{id}` **(new)**	Job status from Redis / RQ	1b
`POST /trips`, `GET /trips/{id}/options`	Create a trip; code-scored options with no Claude call	2
`POST /trips/{id}/optimize`	Hybrid optimizer; validated plan or code fallback	2
`GET /alerts`	Expirations, reconciliation gaps, bonus matches	3

Typed API client from FastAPI's OpenAPI spec (new)

FastAPI publishes a machine-readable description of every endpoint at `/openapi.json`, and the frontend generates its types from it instead of hand-writing them.

1. Add an npm script, `gen:api`, that runs openapi-typescript against `http://localhost:8000/openapi.json` and writes `web/src/api/schema.d.ts`.
2. Create `web/src/api/client.ts`, an openapi-fetch client typed by `schema.d.ts`.
3. Write TanStack Query hooks by hand in `web/src/api/hooks.ts`, around the client.
4. Commit the generated file, and run `npm run gen:api` whenever the API changes.

Rename a field in a Pydantic model, regenerate, and every broken frontend use becomes a compile error. The alternative, **orval**, generates the React Query hooks too: less code to write, but more generated code to understand. openapi-typescript keeps the hooks visible, which suits learning.

Frontend screens

- **Dashboard (1a):** balance cards with the derived balance and last snapshot, portfolio estimate, balance-over-time chart, expiring-soon strip.
- **Ledger (1a):** filterable event table with source, an undo button, and the CSV review queue.
- **Trip planner (2):** trip form, scored options, Claude's plan, transfer simulator.

- **Alerts (3)** and **Settings:** cpp baselines, AwardWallet status, thresholds.

Router decision: React Router vs. TanStack Router

TanStack Router is recommended, because the app already uses TanStack Query and the ledger's filters fit typed search params. Choose React Router if familiarity in other codebases and easy answers online matter more than type safety. The decision is needed in Phase 1a, when the app gets its second screen.

	React Router (v7)	TanStack Router
Type safety	URL params are plain strings unless you use framework mode's type generation	Routes, path params, and search params are fully typed; a link to a missing route is a compile error
Search params	You read and parse `?account=...&kind=...` yourself	Search params are validated by a schema, ideal for ledger filters kept in the URL
Data loading	Loaders in data or framework mode; works alongside TanStack Query	Loaders designed to pair with TanStack Query
Community and docs	The most widely used React router; abundant tutorials and answers	Smaller but active community; fewer answers when you are stuck
Setup	Minimal for a few routes	More upfront: a route tree, often a Vite plugin for file-based routes
Transferable skill	Most likely to match other codebases you join	Teaches type-driven routing, which is increasingly common

Also in the stack: **SQLAlchemy 2.0, synchronous.** FastAPI runs regular functions in a thread pool, and async SQLAlchemy adds hard concepts with no benefit at this scale. **Alembic** handles migrations.

Repo layout (changed)

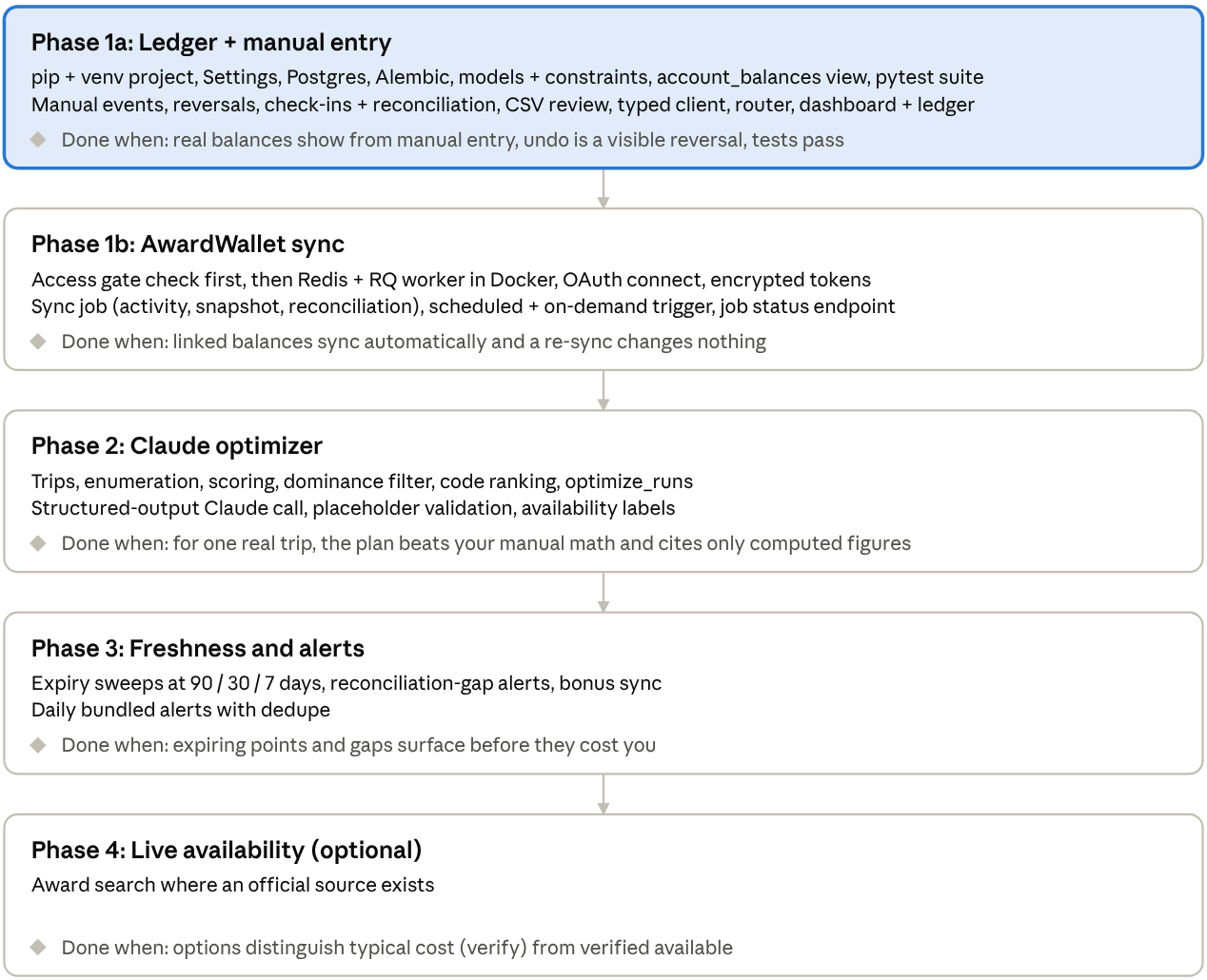
```
Point-Pilot/
  api/
    pyproject.toml, requirements.txt, .venv/ (gitignored)
    app/
      main.py            FastAPI entrypoint   dev  (uvicorn app.main:app --
reload)
      settings.py        pydantic-settings
      db.py              engine + session
      models/            SQLAlchemy models
      routes/            FastAPI routers
      services/          ledger, reconciliation, valuation, optimizer,
                         claude_client, awardwallet
      jobs/
        worker.py        worker entrypoint    python -m app.jobs.worker
        schedule.py      repeating job registration
        sync_awardwallet.py, expiry_sweep.py, bonus_sync.py
    migrations/          Alembic
    tests/               pytest
  web/
    src/api/             generated schema, client, hooks
  docs/
  docker-compose.yml     postgres, redis, worker (API and Vite run natively)
```

Jobs import the same `models`, `services`, and `settings` as the API. Only the entrypoint differs: `main.py` serves HTTP, and `jobs/worker.py` pulls jobs from Redis.

Build order

The app is usable at the end of Phase 1a, from manual entry alone; each later phase adds a source or a feature without changing the ledger.

The app is usable after Phase 1a; each phase ends at a gate



build roadmap · 5 phases, each with a gate

Each phase starts only when the gate above it is met.

Decisions to make before coding

- **Router:** TanStack Router (recommended) or React Router. Needed in Phase 1a.
- **Worker library:** RQ (recommended) or Celery. Needed by Phase 1b.
- **Programs first:** Chase UR, World of Hyatt, and your two most-flown airlines, with starting cpp baselines.
- **Reconciliation tolerance:** the gap size per program that triggers an alert.
- **AwardWallet:** the trust decision and the access gate check, before Phase 1b.
- **Claude failure behavior:** show the code ranking; never block the plan on the model.

First build step: create the `api/` pip project (venv + pyproject.toml) with a `Settings` class, and start Postgres with docker-compose. Then write the first Alembic migration for `programs`, `accounts`, `ledger_events`, and `balance_snapshots` with their constraints, and prove the constraints with pytest.